

FINAL PROJECT: INTEGRATED AI SYSTEM

Smart Solar Panel Monitoring & Maintenance System

Finding which solar inverter is losing energy, why, and what to do about it

Student	[Student Name]
Course	AI Diploma
Instructor	[Instructor Name]
Date	September 2026

Random Forest · ResNet-18 (CNN) · Gemma 2 Transformer · FastAPI · Streamlit

Contents

1. Problem Definition
2. Proposed Solution
3. System Architecture
4. Data Sources
5. Data Preprocessing
6. Numerical Model: Random Forest
7. Image Model: ResNet-18
8. Transformer Chatbot
9. Model Training & Evaluation
10. Results: the System Solving a Real Case
11. API Architecture & Endpoints
12. Streamlit Dashboard
13. Screenshots of the Working System
14. Limitations
15. Future Improvements
16. Conclusion

1. Problem Definition

Solar panels lose output without warning. Dust, bird droppings, snow, cracked glass and electrical faults all reduce the energy a panel produces, but a solar site keeps producing *some* power, so the loss often goes unnoticed. Operators usually find it only when the monthly production report arrives. By then the energy, and the money, is already gone, and the report does not say **which** inverter is affected or **why**.

Deciding whether an inverter is underperforming is also harder than it looks. Output changes all day with sunlight and temperature, so a low reading at 8 a.m. is normal while the same reading at noon is a fault. A fixed threshold cannot tell these apart. A fair judgement needs to know how much the inverter *should* produce under the current weather.

Who uses the system: solar-plant operators and maintenance teams responsible for many inverters and panel strings. They need to know every day which unit needs attention, what the cause is, what to send (a cleaning crew or an electrician) and how urgent it is.

Why AI is appropriate: the expected output of an inverter is a non-linear function of irradiance, temperature and time. A model trained on months of healthy operation learns it far better than hand-written rules. Recognising dust, droppings, cracks or burn marks in a photo is a visual pattern-recognition task, which is what convolutional neural networks do well. Explaining all of this to a non-expert in plain Arabic or English is a natural-language task suited to a Transformer language model.

2. Proposed Solution

The project is **one integrated system** that answers four questions in sequence. Each AI component has exactly one job:

Question	Component	Technique	Output
How much energy is being lost?	Numerical model	Random Forest regression	Expected power, Performance Ratio (PR), kWh lost
Why is it being lost?	Image model	ResNet-18 CNN (transfer learning)	Panel condition (6 classes) + confidence
What should we do, and how urgently?	Decision engine	Rules that fuse both models	Action, responsible team, priority, cost of waiting
Can you explain it to me?	Chatbot	Gemma 2 Transformer (Ollama)	Grounded natural-language answers

Table 1: role of each component

Complete workflow (input to user interaction): inverter and weather readings go to the API. The Random Forest predicts the power each inverter should produce, and the system compares it with the actual power to get a daily Performance Ratio per inverter. Inverters below 90% are flagged Warning and those below 75% Critical. For a flagged inverter, the operator uploads a photo of its panel string. The

ResNet identifies the visible condition, and the decision engine combines *how much* is lost with *why* into a maintenance ticket. Every result is then passed as context to the Gemma chatbot, so the operator can ask what happened and what to do. A live weather forecast (Open-Meteo) is also fed to the Random Forest to predict tomorrow's production, which helps schedule cleaning on low-production days.

3. System Architecture

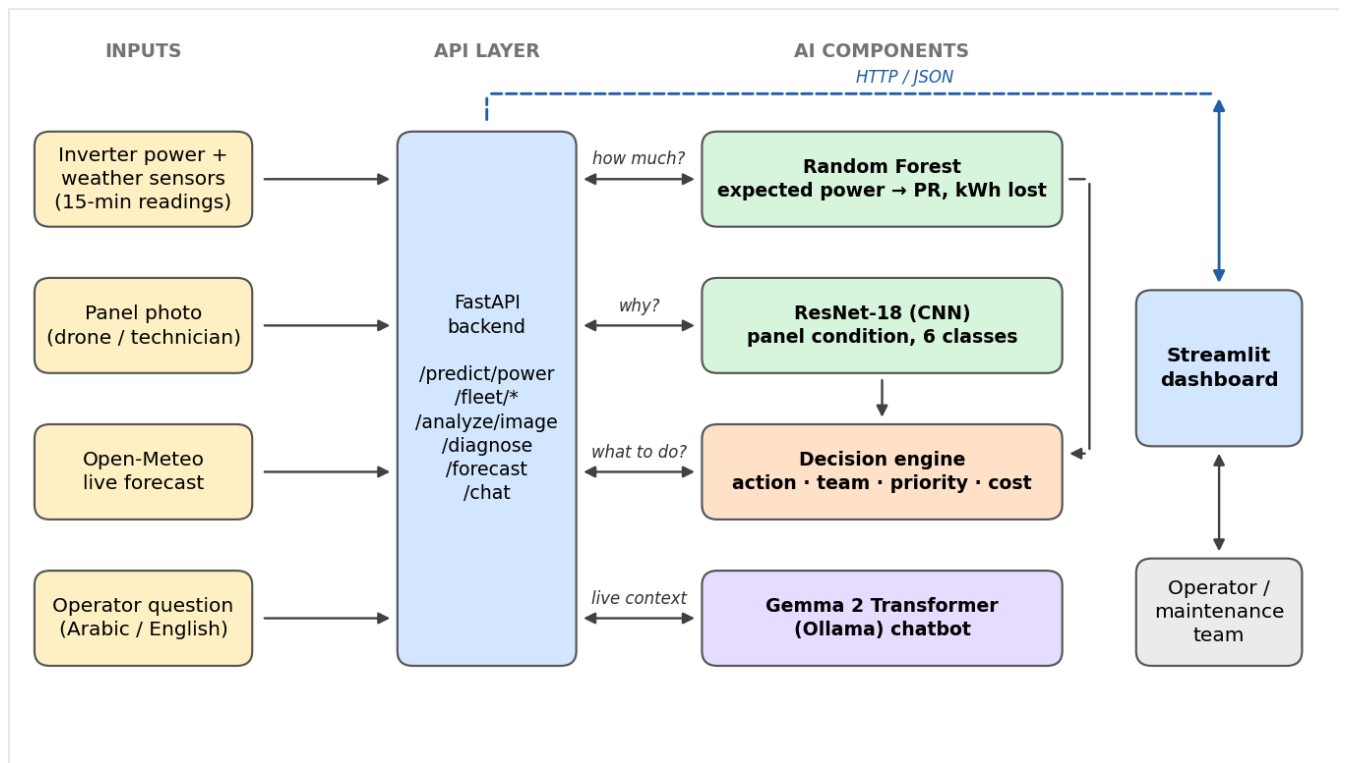


Figure 1: system architecture. Every component communicates through the FastAPI backend.

The architecture has three layers. The **FastAPI backend** is the only place where models are loaded; it exposes each model and the full workflow as REST endpoints. The **AI components** (Random Forest, ResNet-18, decision engine, Gemma) are separate Python modules called by the API. The **Streamlit dashboard** never touches a model directly. It sends HTTP/JSON requests to the API and shows the responses. Because of this separation, the same API could serve a mobile app or an alerting service without changes.

How the models communicate: the Random Forest output (PR and kWh lost) and the ResNet output (class and confidence) are both inputs of the decision engine. The chatbot receives a JSON *context* built by the API from all three: fleet summary, five weakest inverters, selected inverter, latest diagnosis and model accuracy. So the chatbot's answers are always based on the system's current results.

File	Responsibility
src/data_prep.py	Load, clean and merge the Kaggle CSVs; flag healthy rows
src/train_numeric.py	Train Random Forest (+ Linear Regression baseline)
src/train_lstm.py	Train LSTM for comparison on the same test rows
src/train_image.py	Fine-tune ResNet-18 on the GPU
src/monitoring.py	Daily Performance Ratio per inverter
src/vision.py	Image inference
src/decision.py	Maintenance decision engine
src/weather.py	Open-Meteo forecast to expected production
src/chatbot.py	Gemma 2 via Ollama with a grounded system prompt
api/main.py	FastAPI backend, all endpoints
dashboard/app.py	Streamlit dashboard (7 pages)
run.py	Start API + dashboard with one command (python run.py)

Table 2: source-code organisation

4. Data Sources

Dataset	Source	Content	Used by
Solar Power Generation Data	Kaggle: anikannal/solar-power-generation-data	2 plants in India, 44 inverters, 15-min readings for 34 days (15 May to 17 Jun 2020): AC/DC power, irradiation, ambient and module temperature	Random Forest, LSTM, monitoring
Solar Panel Images: Clean and Faulty	Kaggle: pythonafroz/solar-panel-images	885 photos in 6 classes	ResNet-18
Open-Meteo forecast	api.open-meteo.com (free, no key)	Hourly irradiance, temperature and cloud cover for any location	Production forecast

Table 3: data sources

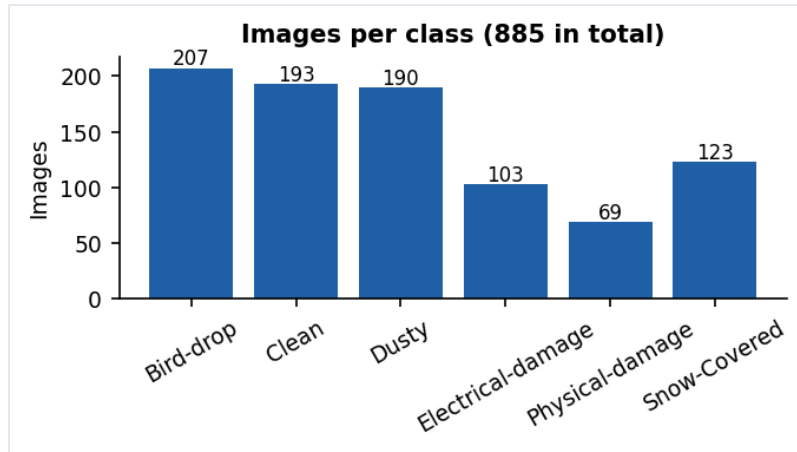


Figure 2: images per class. Physical-damage is the smallest class (69 images).

5. Data Preprocessing

5.1 Numerical data

1. **Merging:** each plant's generation file (one row per inverter per 15 min) was joined with its weather-sensor file on the timestamp. Plant 1 uses a dd-mm-yyyy date format and Plant 2 uses ISO, so both were parsed explicitly. The result has 136,472 rows.
2. **Cleaning:** removed missing values, duplicate (timestamp, plant, inverter) rows and negative power or irradiation.
3. **Feature engineering:** hour of day encoded as sine and cosine (so 23:45 and 00:00 are close), plus the plant id. The final features are irradiation, ambient temperature, module temperature, HOUR_SIN, HOUR_COS and PLANT. The target is AC power (kW).
4. **Healthy-operation filter (key step):** the model must learn what a *healthy* inverter produces. If faulty readings were included it would learn faults as normal and could not detect them. Excluded from training: (a) daylight outages (irradiation above 0.05 but zero power), (b) momentary faults where an inverter produces under 60% of its plant's median inverter at the same timestamp, and (c) chronic under-performers, meaning inverters whose long-run output is under 85% of the plant median. This left 126,356 healthy rows. Faulty rows are kept for monitoring, where the system must find them.
5. **Chronological split:** the last 7 days (11 to 17 June) are the test set (27,653 rows) and the first 27 days the training set (98,703 rows). This mirrors real use: train on the past, predict the future.

5.2 Image data

1. Corrupted files were skipped by verifying every image with Pillow; all images were converted to RGB.
2. **Stratified split** 70 / 15 / 15: 619 train, 133 validation and 133 test images, with the same class proportions in each split.
3. **Augmentation (training only):** random resized crop (60 to 100% of the image), horizontal and vertical flips, rotation of $\pm 15^\circ$ and colour jitter. This simulates different camera angles, distances and lighting.
4. Resized to 288×288 and normalised with the ImageNet mean and standard deviation, as the pretrained network expects.

5. **Class weights** in the loss function counter the imbalance (Physical-damage has 69 images, Bird-drop has 207).

6. Numerical Model: Random Forest

A **Random Forest Regressor** (scikit-learn) predicts the AC power an inverter should produce from the six features above. It averages many decision trees trained on bootstrap samples. It captures non-linear effects such as the power drop at high module temperature and inverter clipping at peak sun, needs no feature scaling, trains in seconds (3.3 s) and gives feature importances that make it easy to explain.

Hyper-parameters: 150 trees, max_depth = 18, min_samples_leaf = 10, all CPU cores. Limiting depth and leaf size keeps the model small (17 MB) and reduces over-fitting.

Role in the system: Performance Ratio **PR** = **actual energy** ÷ **expected energy**, computed per inverter per day over daylight readings. PR ≥ 90% is Normal, 75 to 90% Warning and below 75% Critical. The expected minus actual energy is the energy lost, which the decision engine converts to money.

Model selection: two alternatives were trained on the same data. Linear Regression serves as a baseline. An **LSTM** (2 layers, 64 hidden units) sees the previous 2 hours (8 × 15 min) of weather rather than a single reading; it was trained on the RTX 5060 GPU in 9.1 s. All models are scored on the same test rows (Section 9).

7. Image Model: ResNet-18

ResNet-18 is a convolutional neural network whose residual (skip) connections let gradients flow through deep layers. We use **transfer learning**: the network was pre-trained on ImageNet (1.2 million images), so its early layers already detect edges, textures and shapes. Only the final classifier was replaced with a new 6-class head (dropout 0.3 + linear layer). Training from scratch on 885 images would over-fit badly; transfer learning reuses general visual knowledge.

Two-phase training on an NVIDIA RTX 5060 Laptop GPU with mixed precision (FP16):

- **Phase 1: head only** (4 epochs, learning rate 1e-3). The backbone is frozen so the new random layer does not damage the pretrained features.
- **Phase 2: full fine-tuning** (26 epochs, learning rate 3e-4, cosine schedule). All layers adapt to solar-panel images.
- AdamW optimiser (weight decay 1e-4), batch size 32, class-weighted cross-entropy with label smoothing 0.05. The checkpoint with the best validation accuracy is kept.

Role in the system: it tells the operator *why* an inverter is losing energy. Its output class chooses the maintenance action (cleaning crew, electrician or panel replacement), and electrical and physical damage are treated as safety issues.

8. Transformer Chatbot

The chatbot is **Gemma 2**, Google's open Transformer language model, running **locally** through Ollama. No data leaves the machine and no paid API is needed. The system uses gemma2:9b when it is installed and falls back to gemma2:2b (configurable in src/config.py). The screenshots in this report were taken with gemma2:2b.

Integration, not a separate component: the chatbot is never asked a bare question. For every message, the API builds a JSON context from the live system: fleet summary for the selected day, the five weakest inverters (worst first), the selected inverter, the latest maintenance ticket and the accuracy of both models. The system prompt describes the three AI components and the PR thresholds. It also instructs the model to answer **only** from this data, never invent numbers, and reply in the user's language. Answers are streamed token by token to the dashboard.

Reliability measures: small language models misread raw ratios. In early tests gemma2:2b described PR = 0.859 as "85.9% loss" and picked the wrong worst inverter from an unsorted list. The API now sends numbers pre-computed in plain words (performance_percent, energy_lost_percent, worst_inverter_today, list sorted worst first), and Arabic questions get an explicit "answer in Arabic" instruction. After these changes the answers were correct in both languages (Figure 14).

Retrieval step for Arabic questions: after a diagnosis, the small model tended to repeat the urgent maintenance action even when asked something else. A light retrieval step now reads the question, detects what it asks about (energy lost, the worst inverter, the number of critical inverters, or what to do) and passes only those facts, written in Arabic, next to the question. The API also builds a ready-made answer to "what should we do?" (what_to_do_now) from the latest ticket, which is kept on the server so a page refresh does not lose it. With a fixed seed and temperature 0, the tested Arabic questions were answered correctly and consistently.

9. Model Training & Evaluation

9.1 Numerical model

Model	MAE (kW)	RMSE (kW)	R ²
Linear Regression (baseline)	24.1	54.7	0.972
Random Forest	17.2	53.7	0.973
Random Forest, daylight rows only	37.0	79.2	0.929

Table 4: test set, last 7 days (27,653 rows)

Model (same 27,146 test rows)	MAE (kW)	RMSE (kW)	R ²
Linear Regression	23.4	52.6	0.974
LSTM (8-step weather history)	23.2	65.3	0.961
Random Forest	16.5	51.1	0.976

Table 5: all three models on identical test rows (rows with a full 2-hour history)



Figure 3: the three numerical models on identical test rows. The Random Forest has the lowest error.

The Random Forest explains **97.3%** of the variance in inverter power. On identical test rows its average error is **29% lower** than Linear Regression (16.5 vs 23.4 kW) and clearly lower than the LSTM. Linear Regression is close on R² because power is almost proportional to irradiance. The Random Forest also captures the non-linear parts (clipping at peak sun, heat losses), which reduces the typical error. The extra history does not help the LSTM because power depends on the *current* irradiance, not the past hours. The Random Forest was therefore chosen: it is the most accurate, trains in seconds and is easy to explain. The daylight-only score (R² 0.929) is the honest measure, since night-time rows are trivially zero.

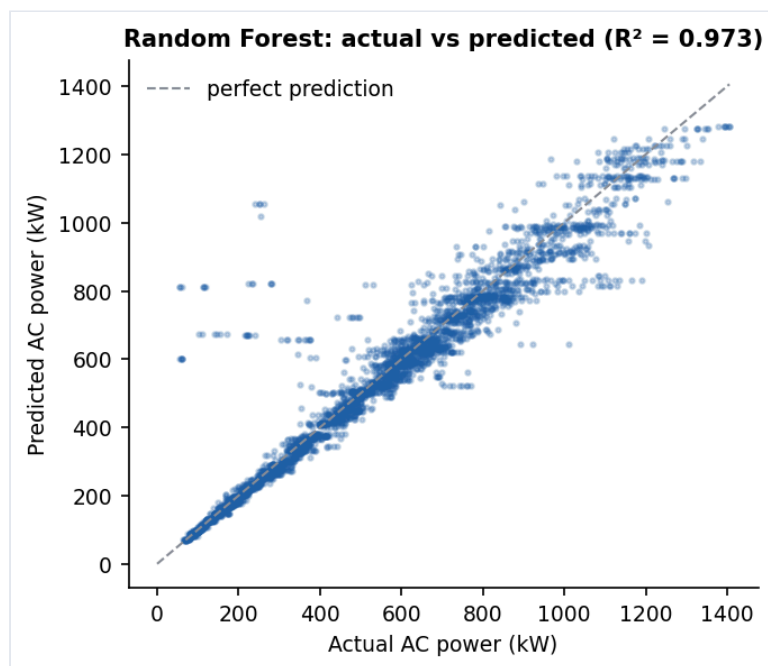


Figure 4: actual vs predicted power on test days (daylight). Points lie close to the diagonal.

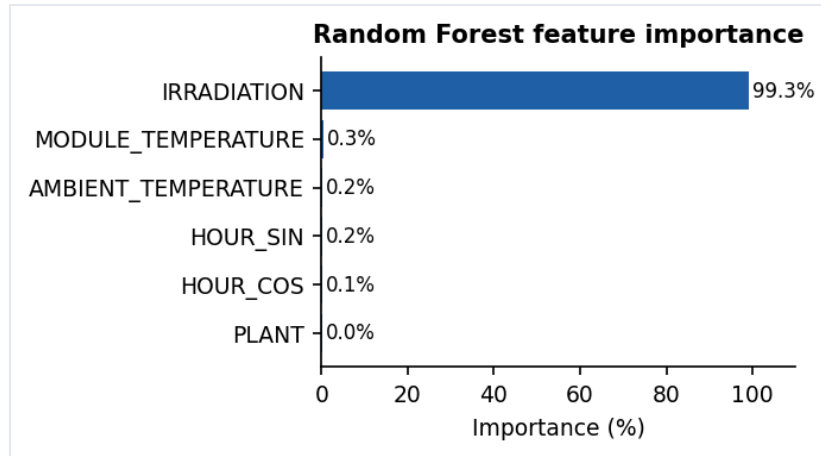


Figure 5: feature importance. Irradiation dominates, as the physics of solar panels predicts.

9.2 Image model

Class	Precision	Recall	F1-score	Test images
Bird-drop	0.93	0.87	0.90	31
Clean	0.85	1.00	0.92	29
Dusty	0.89	0.83	0.86	29
Electrical-damage	0.93	0.93	0.93	15
Physical-damage	0.90	0.82	0.86	11
Snow-Covered	0.94	0.94	0.94	18
Macro average	0.91	0.90	0.90	133

Table 6: ResNet-18 on the unseen test set, accuracy 90.2%

	Bird-drop	Clean	Dusty	Electrical-damage	Physical-damage	Snow-Covered
Bird-drop	27	0	3	1	0	0
Clean	0	29	0	0	0	0
Dusty	1	4	24	0	0	0
Electrical-damage	0	0	0	14	0	1
Physical-damage	1	1	0	0	9	0
Snow-Covered	0	0	0	0	1	17

Figure 6: confusion matrix on the 133 test images.

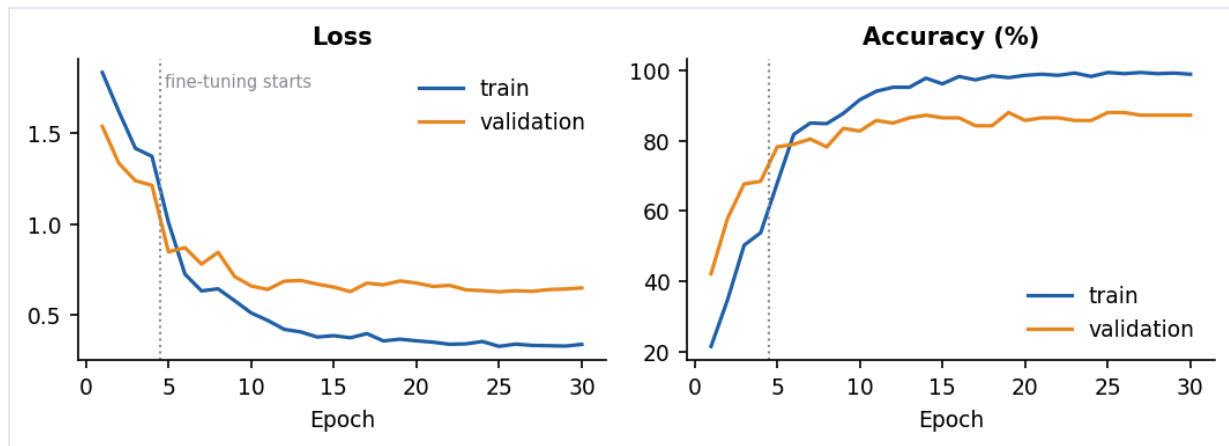


Figure 7: training curves. Validation accuracy jumps once fine-tuning starts (dotted line).

The model reaches **90.2% accuracy** and a macro F1-score of **0.902** on test images it never saw, after 4 minutes of GPU training. All 29 Clean panels were recognised. The most frequent error is Dusty predicted as Clean (4 of 29): a thin dust layer is hard to see in some photos. **Experiment:** the first run (224 px images, 16 fine-tuning epochs, learning rate 1e-4) reached 84.2%. Raising the resolution to 288 px and fine-tuning longer at 3e-4 gave the final 90.2%, because small defects such as droppings and cracks are more visible at higher resolution.

10. Results: the System Solving a Real Case

Running the full system over the 34 days produced 1,464 inverter-days: **1,155 Normal, 70 Warning and 239 Critical**. Most Critical cases are in Plant 2, whose inverters repeatedly dropped out in the middle of the day.

Worked example (20 May 2020): the fleet produced 216,409 kWh while the Random Forest expected 262,188 kWh, so the site ran at **82.5%** and lost **48,344 kWh** (about **\$4,834** at \$0.10/kWh) in one day. Ten inverters were Critical.

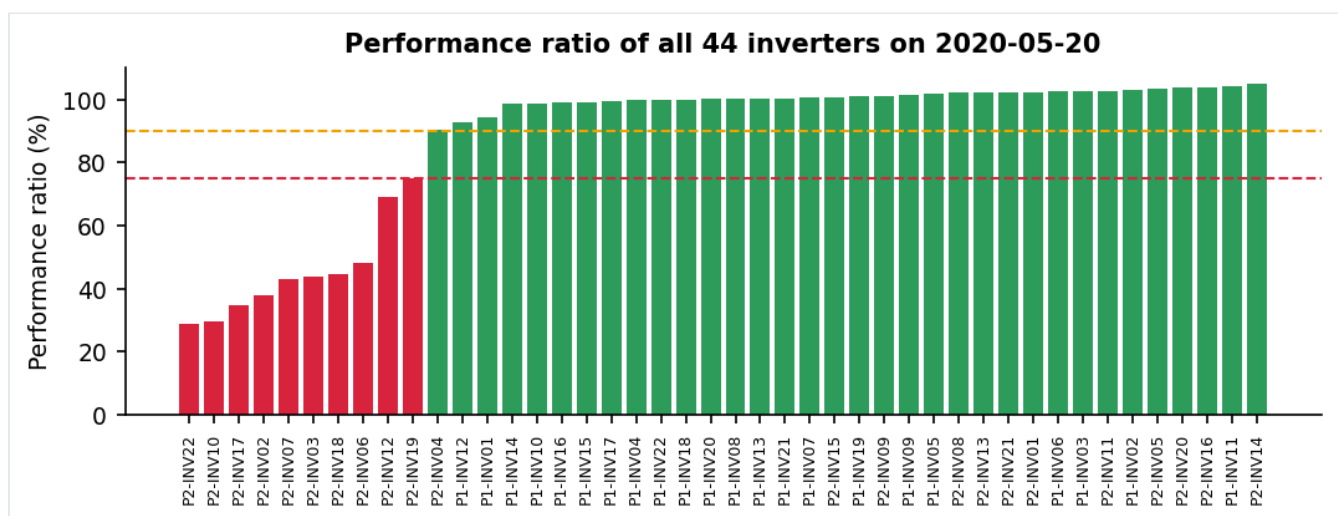


Figure 8: Performance Ratio of all 44 inverters on 20 May 2020. Ten are below the 75% Critical line.

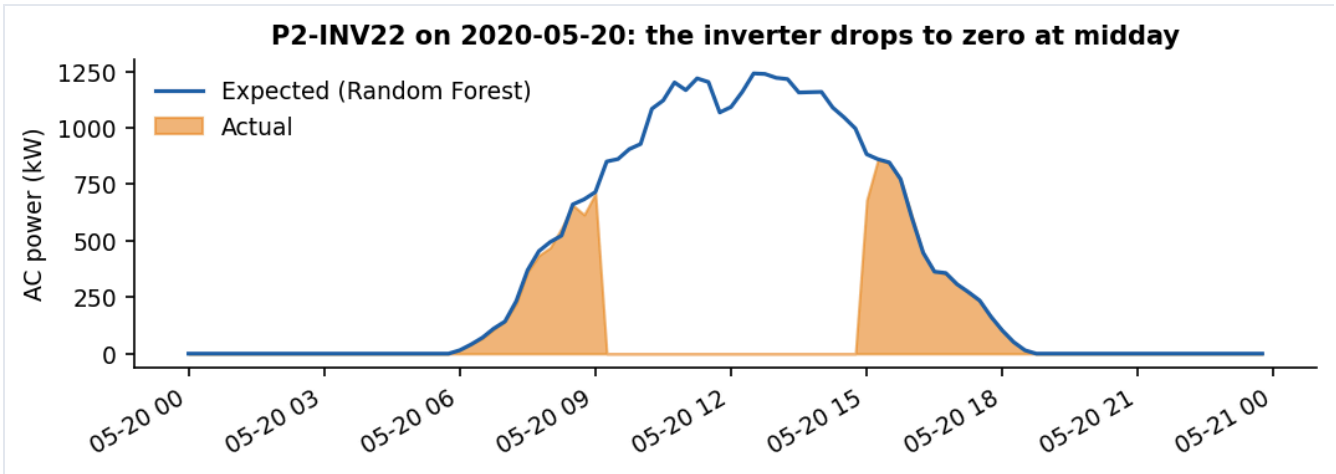


Figure 9: the worst inverter, P2-INV22, matches the expected curve in the morning and then drops to zero from about 09:00 to 15:00.

Step	Component	Result for P2-INV22
1. How much?	Random Forest	PR 28.7%, Critical; 2,588 kWh produced vs 9,012 kWh expected; 6,424 kWh lost
2. Why?	ResNet-18	Panel photo classified as Electrical-damage (97% confidence)
3. What to do?	Decision engine	Priority URGENT. Isolate the string and send an electrician immediately (fire risk). Cost \$642 per day, \$4,497 if ignored for a week
4. Explain	Gemma 2	Answered in English and Arabic that P2-INV22 is the worst (28.7%, 6,424.5 kWh lost) and that an electrician must be sent because of the fire risk

Table 7: end-to-end workflow on a real case

The decision engine gives consistent outcomes across conditions. A clean panel at 99.9% gets "no action needed". A dusty panel at 75% gets a High-priority cleaning job. Electrical or physical damage is never ranked below its safety level, and only safety issues can reach Urgent.

11. API Architecture & Endpoints

The backend is built with **FastAPI**. Input is validated with Pydantic, for example irradiation must be between 0 and 1.5 kW/m² and hour between 0 and 24. Interactive documentation is generated automatically at /docs (Figure 17). Models are loaded once and cached, so each request only runs inference.

Method	Endpoint	Purpose
GET	/health	Status of API, models, GPU and chatbot model
POST	/predict/power	Weather reading → expected power; with actual power also PR + status
GET	/fleet/dates, /fleet/summary	Available days; daily fleet KPIs
GET	/fleet/inverters	PR, energy and status for every inverter on a day
GET	/fleet/plant_curve	Plant actual vs expected power curve
GET	/fleet/inverter/{name}	Day curve and full PR history of one inverter
POST	/analyze/image	Panel photo → class + probabilities of all 6 classes
POST	/diagnose	Inverter + photo → maintenance ticket (full workflow)
GET	/forecast	Open-Meteo forecast → expected production today and tomorrow
POST	/chat/context	Shows the exact context the chatbot receives
POST	/chat	Question + live context → Gemma answer (streamed)
GET	/metrics, /metrics/*	Evaluation results of all models
GET	/demo/images, /demo/image/{id}	Unseen test photos for live demos

Table 8: API endpoints

Example: POST /predict/power with irradiation 0.8 kW/m², 30 °C ambient, 50 °C module, 12:00, plant 1 and actual 600 kW returns expected 1,041.7 kW, PR 0.576 and status Critical.

12. Streamlit Dashboard

The Streamlit dashboard is the main interface and has seven pages. The sidebar holds the monitoring-day selector and a live **system status** panel (API, Random Forest, ResNet, chatbot model, GPU), filled from the /health endpoint.

Page	What it shows
Overview	Problem, solution, fleet KPIs for the day, architecture diagram
Fleet Monitoring	Plant actual vs expected curves, PR per inverter, sortable table, inverter drill-down, interactive prediction form
Panel Inspection	Upload a photo or use an unseen test photo, then see the class and probability of each class
Diagnosis & Maintenance	The full workflow: inverter selection (worst first), photo, and the maintenance ticket with priority and cost
Production Forecast	Live Open-Meteo forecast for any latitude/longitude and the expected hourly production
AI Assistant	Gemma chat with example questions and a view of the context it receives
Model Performance	Metrics, model comparison, confusion matrix, training curves, endpoint list

Table 9: dashboard pages

13. Screenshots of the Working System

All screenshots were captured automatically from the running system (API + dashboard) while it performed the demo flow described in Section 10.

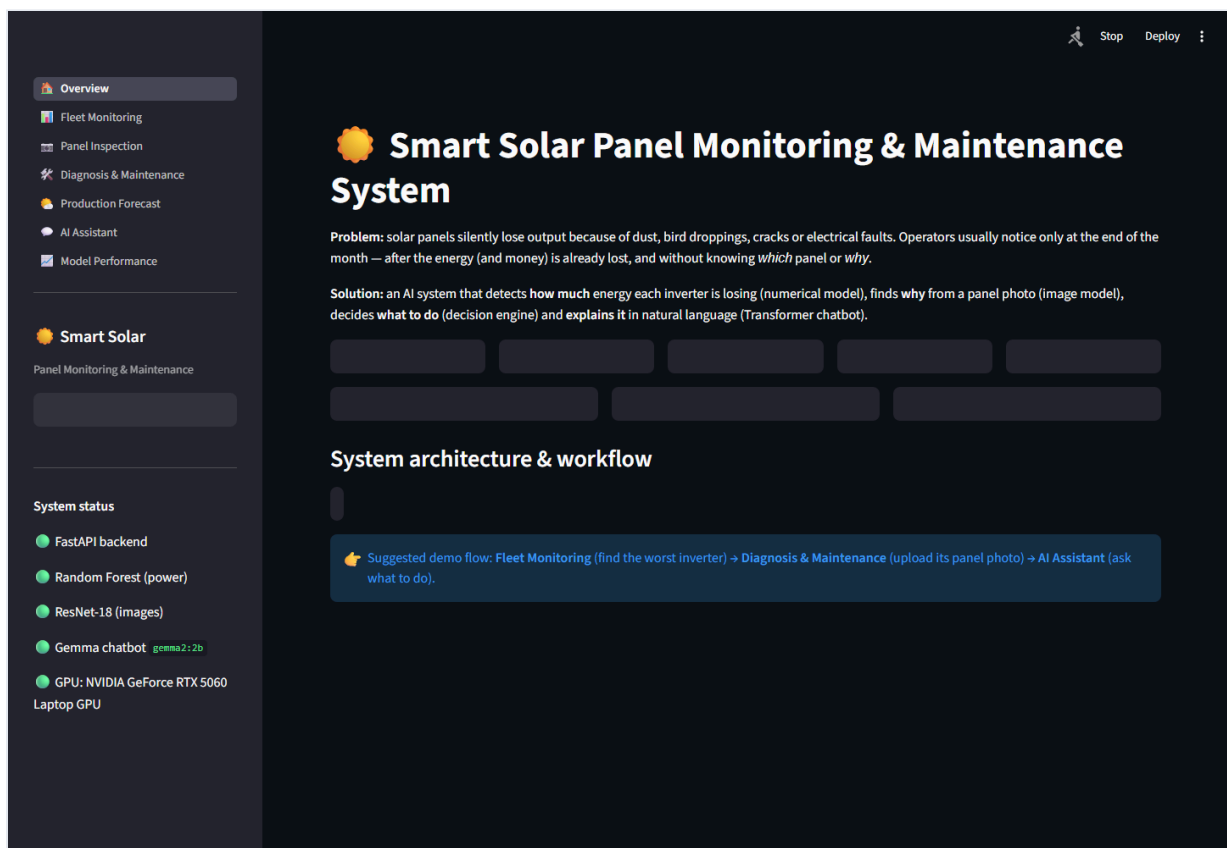


Figure 10: Overview page with fleet KPIs and architecture

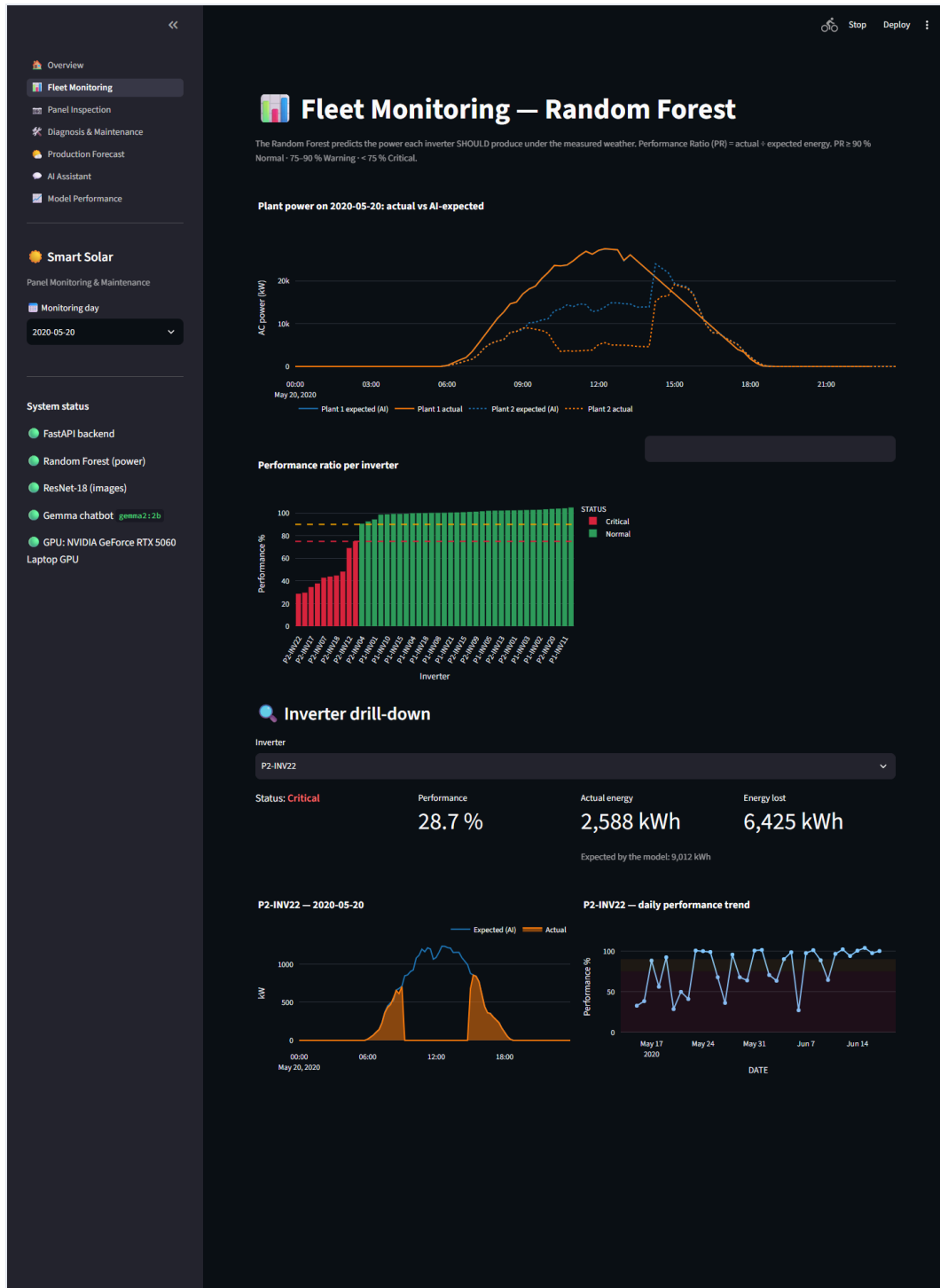


Figure 11: Fleet Monitoring (Random Forest). P2-INV22 drill-down shows the midday outage

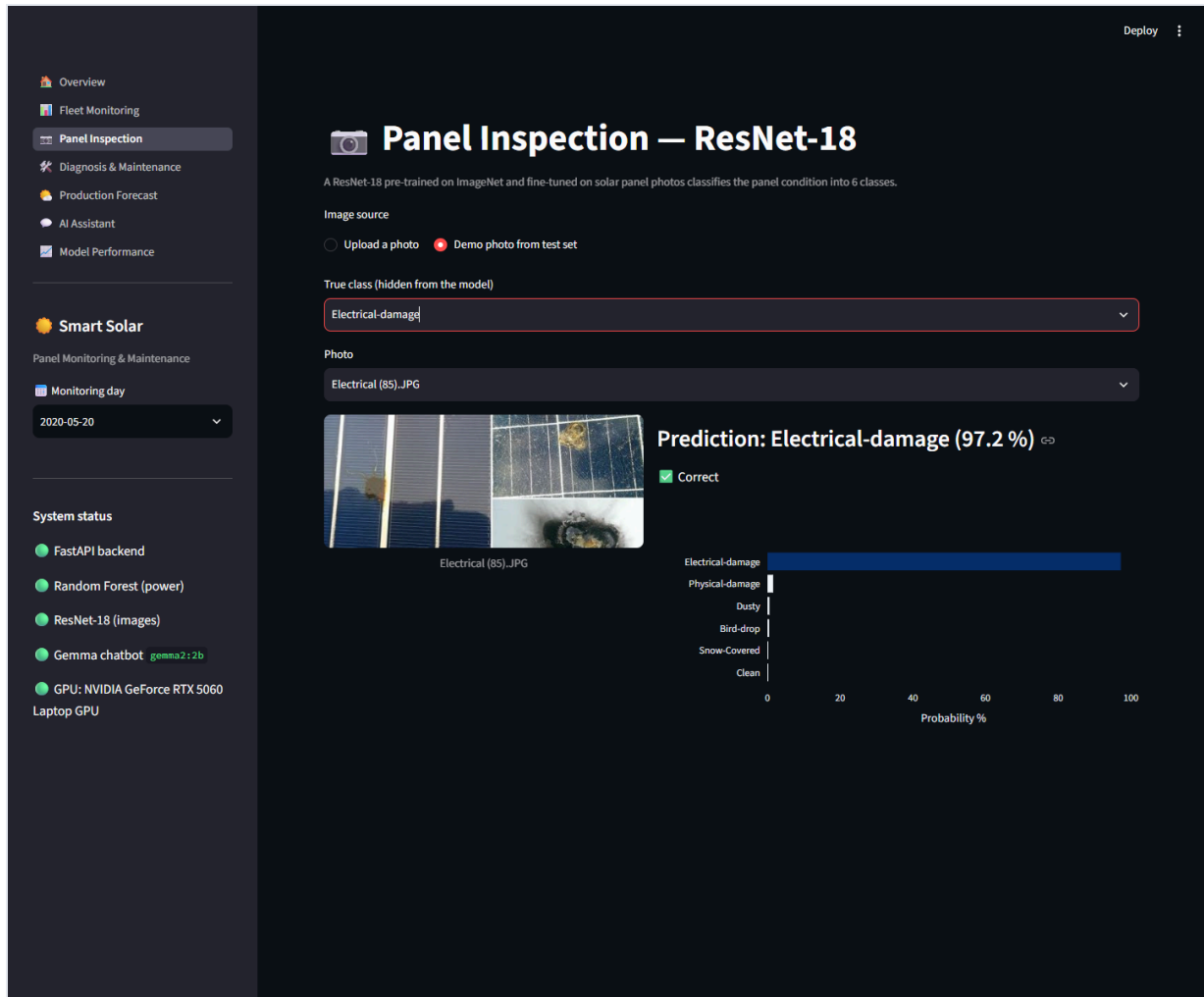


Figure 12: Panel Inspection (ResNet-18) on an unseen Electrical-damage photo

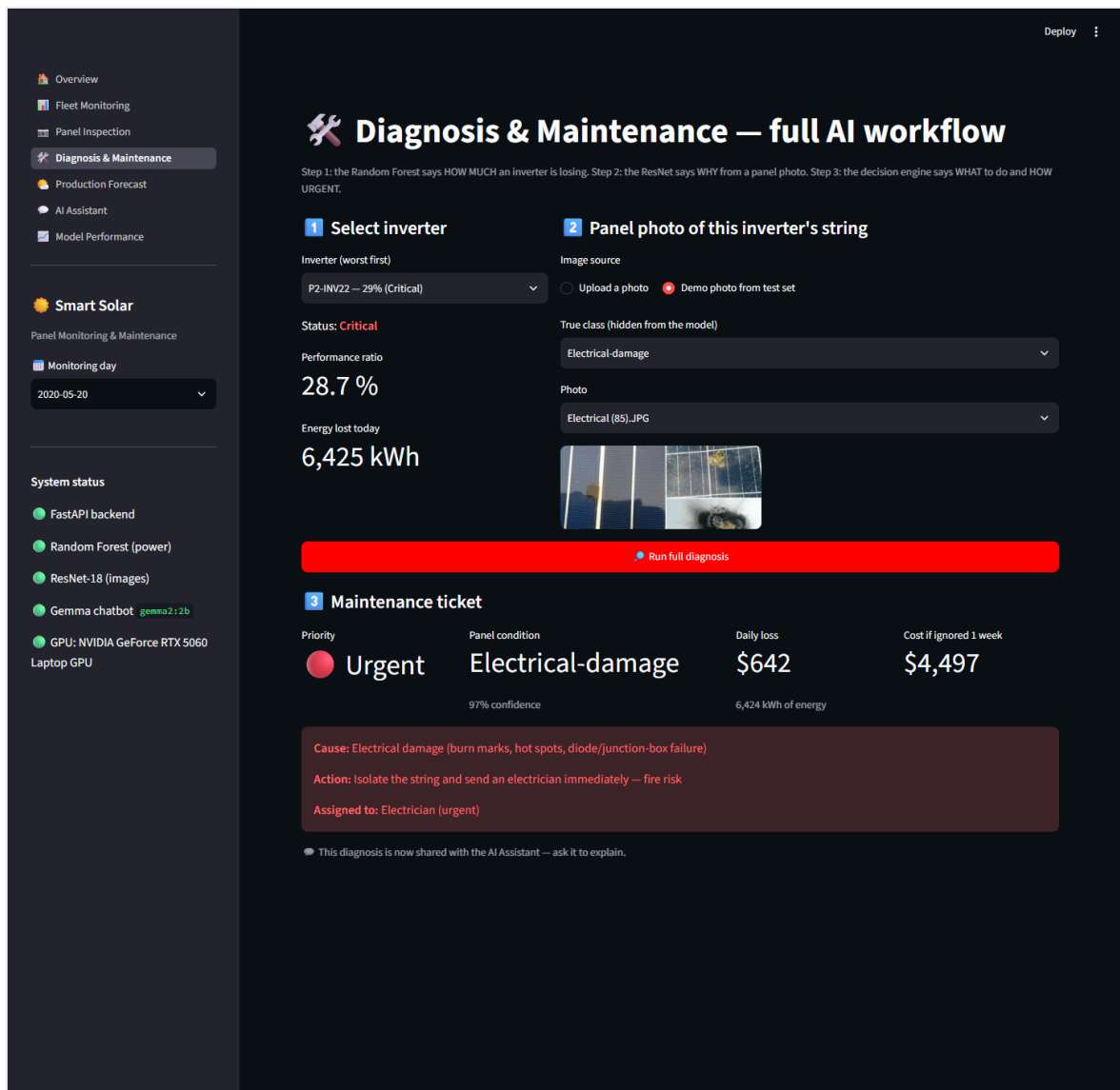


Figure 13: Diagnosis & Maintenance. Full workflow producing an URGENT ticket

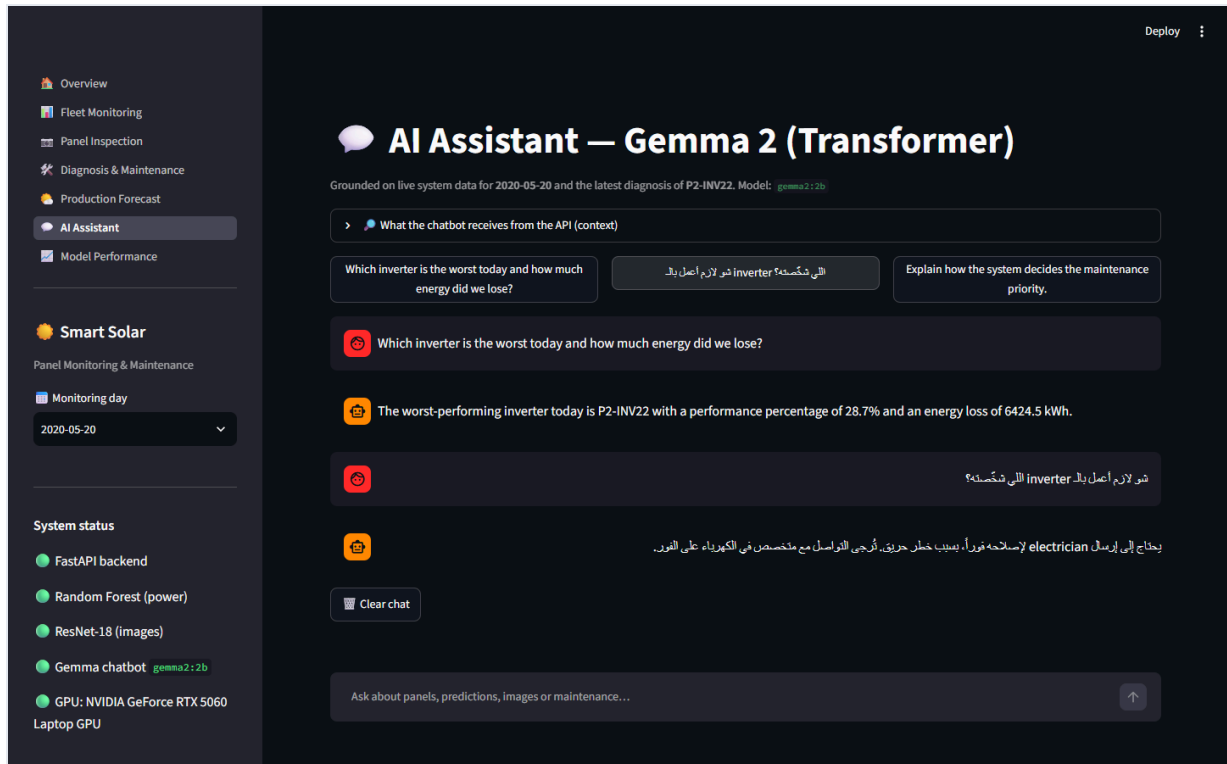


Figure 14: AI Assistant (Gemma 2) answering from live system data in English and Arabic

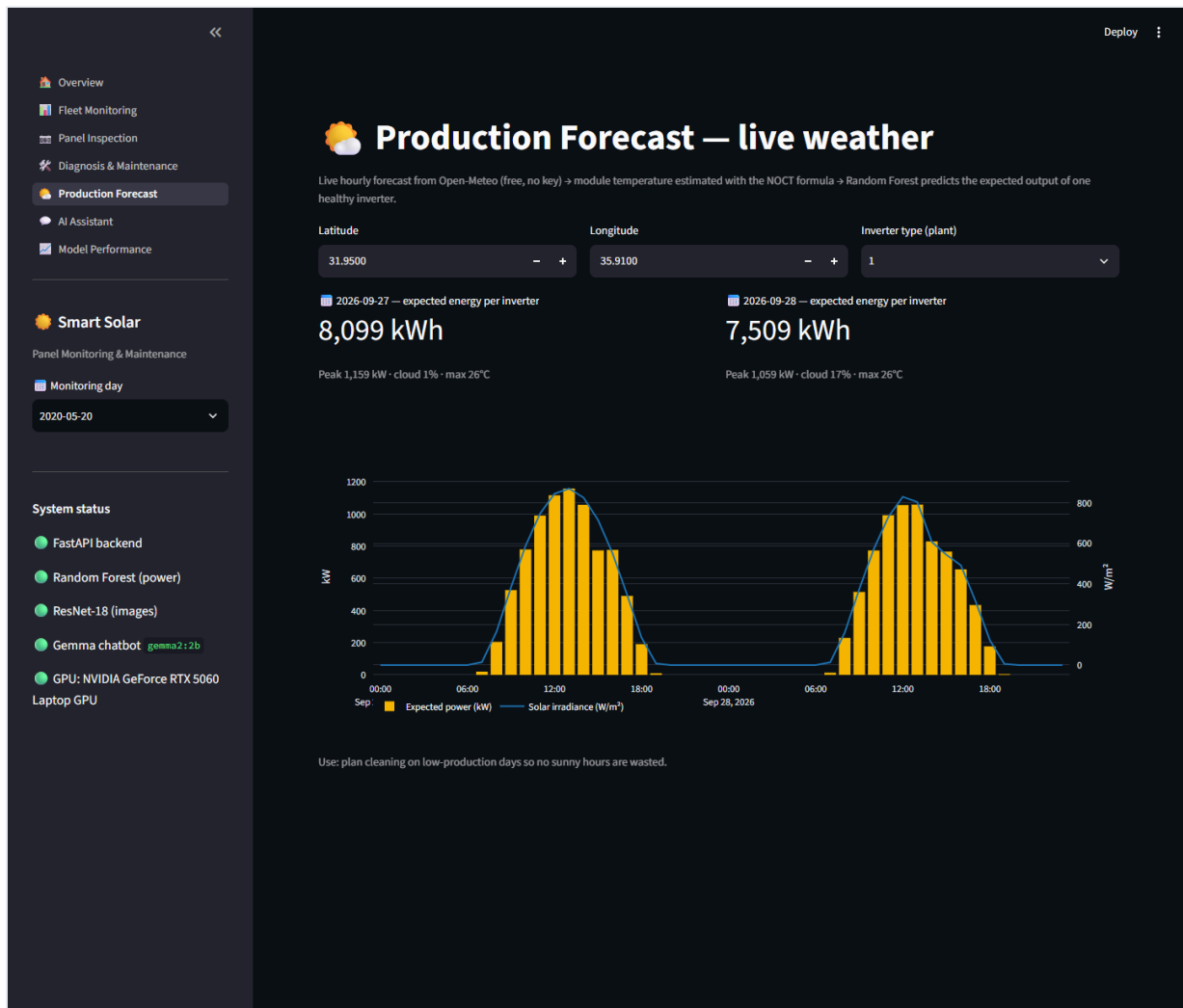


Figure 15: Production Forecast from live Open-Meteo weather



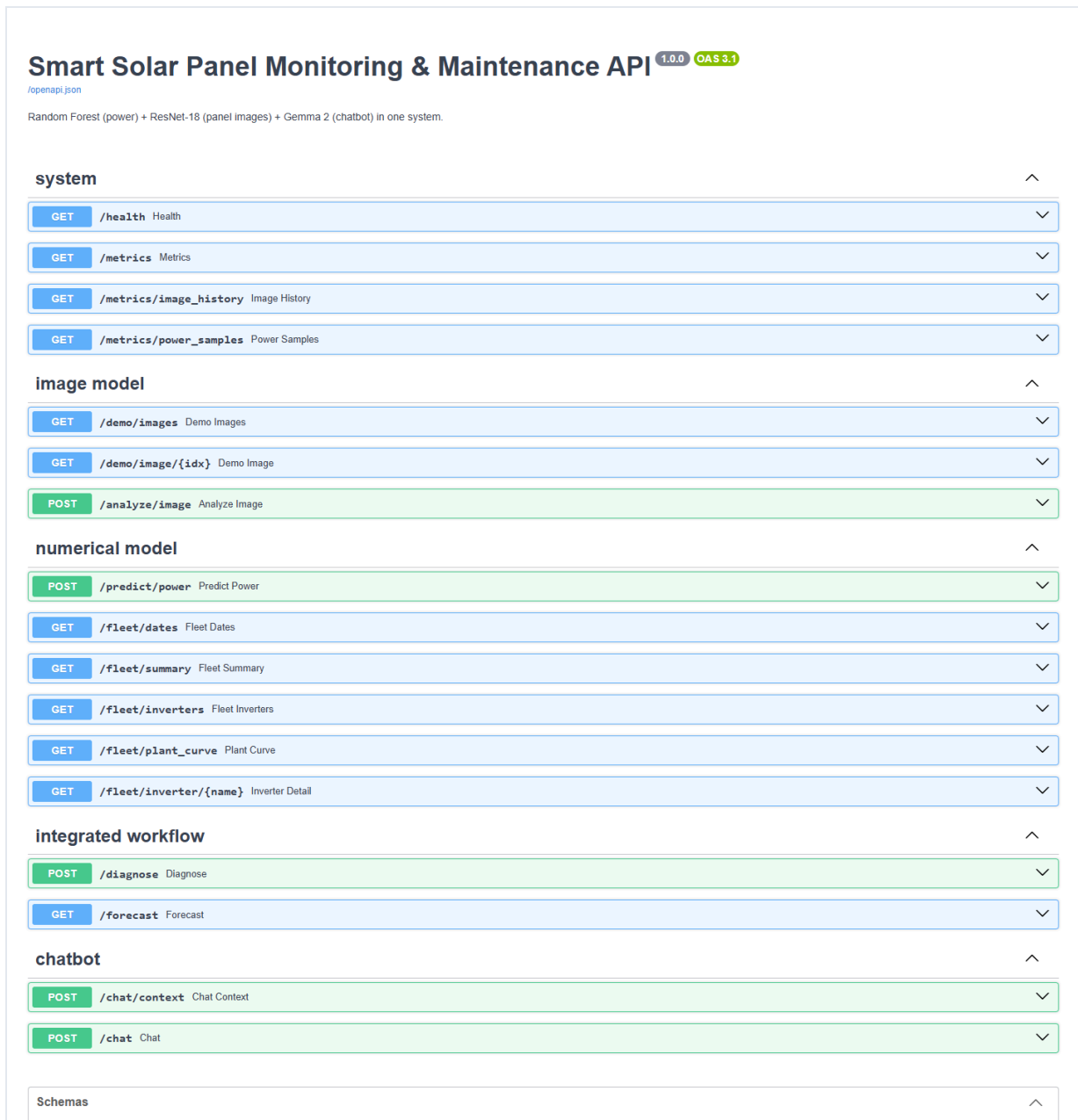


Figure 17: FastAPI interactive documentation (/docs)

14. Limitations

- **Short, single-season data:** 34 days from two plants in India, one season. Winter conditions, panel ageing and other climates are not represented.
- **Images and inverters are not paired:** the two Kaggle datasets are independent, so the photo of a flagged inverter's string is supplied by the operator. In a real site a drone would photograph each string.
- **Plant-level weather sensors:** there is one irradiance sensor per plant, so local shading of a single string cannot be separated from a fault.
- **Small image dataset:** 885 photos of mixed quality; Physical-damage has only 69. Light dust is sometimes confused with Clean.

- **Rule-based decisions:** priorities and the \$0.10/kWh tariff are fixed rules, not learned from maintenance history.
- **Local language model:** gemma2:2b occasionally misreads numbers (mitigated by the pre-computed context); gemma2:9b is better but needs about 6 GB of GPU memory.
- **Forecast transfer:** the Random Forest trained on Indian plants is applied to other locations' forecasts, which assumes similar panels and inverters.

15. Future Improvements

- Automatic drone flights that photograph every string and link each photo to its inverter.
- Object detection (e.g. YOLO) and thermal (infrared) images to locate the exact faulty cell and detect hot spots invisible to normal cameras.
- Train on a full year and more sites; add panel age and soiling rate as features.
- Time-series anomaly detection to catch gradual degradation before PR crosses a threshold.
- Automatic alerts (SMS, e-mail, WhatsApp) to the responsible team when a ticket is Urgent.
- Optimise the cleaning schedule with the forecast: clean on cloudy days so no sunny hours are lost.
- Deploy with Docker on a server, add user accounts, and keep a history of tickets to learn priorities from real outcomes.

16. Conclusion

The project answers its main question: **what real-world problem does the system solve, and how do its components work together?** It solves hidden energy loss on solar sites by telling the operator, on the same day, *which* inverter is losing energy, *how much*, *why*, *what to do* and *how urgent* it is, and it explains all of this in natural language.

Each component has one clear purpose. The Random Forest (R^2 0.973) turns weather into an expected-power baseline that exposes under-performance. The ResNet-18 (90.2% accuracy) finds the visible cause. The decision engine turns both into an action with a priority and a cost. The Gemma 2 Transformer explains the live results. FastAPI connects everything, and Streamlit presents it as one product. On real data the system found an inverter losing 6,424 kWh a day, diagnosed an electrical fault and raised an urgent, safety-first ticket, going from raw data to a decision a maintenance team can act on.